University of Glasgow

WORLD CHANGING GLASGOW

电子科技大学 格拉斯哥学院 Glasgow College, UESTC

电子科技大学 格拉斯哥海南学院 Glasgow College Hainan, UESTC

Lecture 3
Analogue-to-Digital Converters (ADCs)

UESTC 2004: Embedded Processors

Dr Mark D. Butala

Some slides were prepared by Dr Yao Sun, Dr Lina Mohjazi, and Dr Ahmad Taha

Not everything covered in lecture “will be on the exam”

Refer to the chili pepper scale:

- 🌶️ : Supplemental (unlikely to appear on the exam)
- 🌶️🌶️: Just for fun (not on the exam)
- Otherwise: …

Lesson Plan

Aim

- Use the ADC channels on the NUCLEO L432KC to interface with analogue input devices and implement External, Timeout, and Ticker Interrupts

Learning Objectives

By the end of this lesson you will be able to:

- Recall the characteristics and working principle of an ADC
- Write C++ code to interface analogue input devices
- Interrupt implementation on the mbed

Assumed Previous Knowledge

- Basic MCU interfacing techniques
- C programming

Click for Course Specifications & Learning Outcomes

Some Notation (🌶️)

We will use the following symbols:

- $\mathbb{N}$ (**自然数**): the set of natural numbers, i.e., $\mathbb{N} \triangleq \{1, 2, \dots\}$
- $\mathbb{Z}$ (**整数**): the set of integers, i.e., $\mathbb{Z} \triangleq \{\dots, -2, -1, 0, 1, 2, \dots\}$
- $\mathbb{Q}$ (有理数): the set of rational numbers, i.e.,

$$\mathbb{Q} \triangleq \left\{\frac{a}{b}, a \in \mathbb{Z}, b \in \mathbb{N}\right\}$$

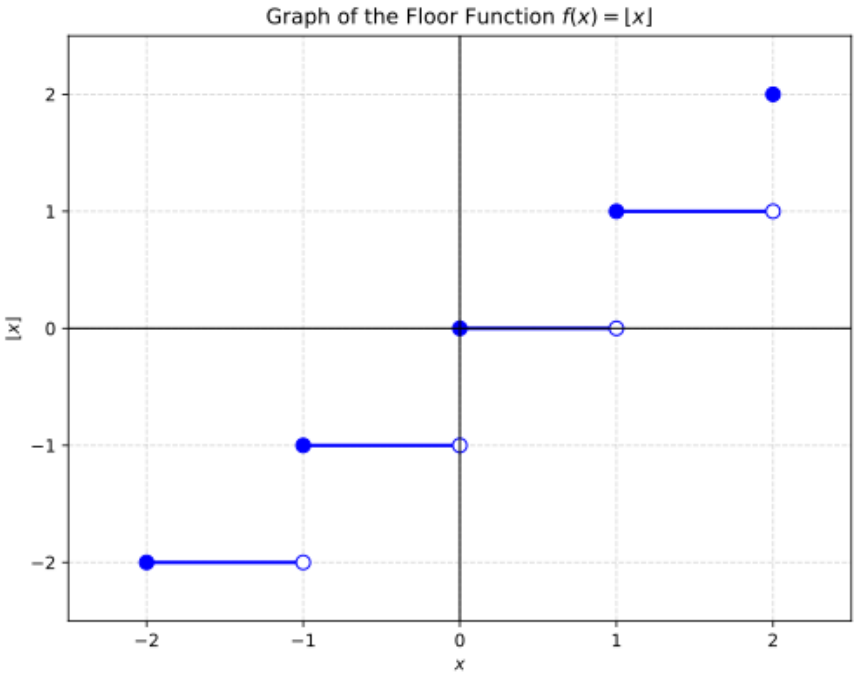
- $\mathbb{R}$ (实数): the set of real numbers

Clearly, $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$. ($\mathbb{N}$ **包含于** $\mathbb{Z}$, ...)

Some Notation (🌶️)

We will soon consider the “floor” function (向下取整函数):

$$\lfloor x \rfloor \triangleq \max\{z \in \mathbb{Z} \mid z \leq x\}$$



Some Definitions ( )

Three types of signals considered this (and next) week:

1. Analogue (GB) / analog (US) signals
   - Continuous both in time and value
   - Example:

$$x(t) = A\cos(2\pi f_0 t + \phi)$$

   - Note that $t \in \mathbb{R}$ and $x(t) \in \mathbb{R}$

Some Definitions ( )

Three types of signals considered this (and next) week:

2. Discrete-time signals

- Discrete in time and continuous in value
- Example:

$$x[n] = A\cos(2\pi f_0 T n + \phi)$$

- Note that $n \in \mathbb{Z}$ and $x[n] \in \mathbb{R}$

Some Definitions ( )

Three types of signals considered this (and next) week:

3. Digital signals
   - Discrete in time and rational in value
   - Example:

$$x_d[n] = \lfloor A\cos(2\pi f_0 T n + \phi) \rfloor$$

where $\lfloor z \rfloor$, i.e., the floor of z, is the greatest integer less than or equal to z

- Note that $n \in \mathbb{Z}$ and $x_d[n] \in \mathbb{Q}$
- Each value of $x_d[n]$ can be one-to-one mapped to a sequence of bits

Reality is Analog, Computers are Digital

Analog input from sensors:

- Temperature
- Audio
- Accelerometer
- Light intensity
- …

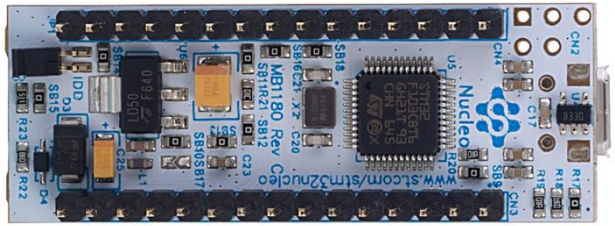
Digital processing on MCU

Reality is Analog, Computers are Digital

Analog input from sensors:

- Temperature
- Audio
- Accelerometer
- Light intensity
- …

Digital processing on MCU



How to bridge the gap between analog input signals (of voltage) and digital processing?

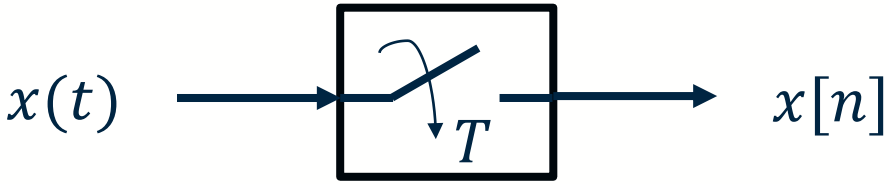
Analog to Digital Converter

$t \in \mathbb{R}, x(t) \in \mathbb{R}$ $\qquad$ $n \in \mathbb{Z}, x_d[n] \in \mathbb{Q}$

An analog to digital converter (ADC or A/D) has two stages:

1. Sampling
2. Quantization

Sampling



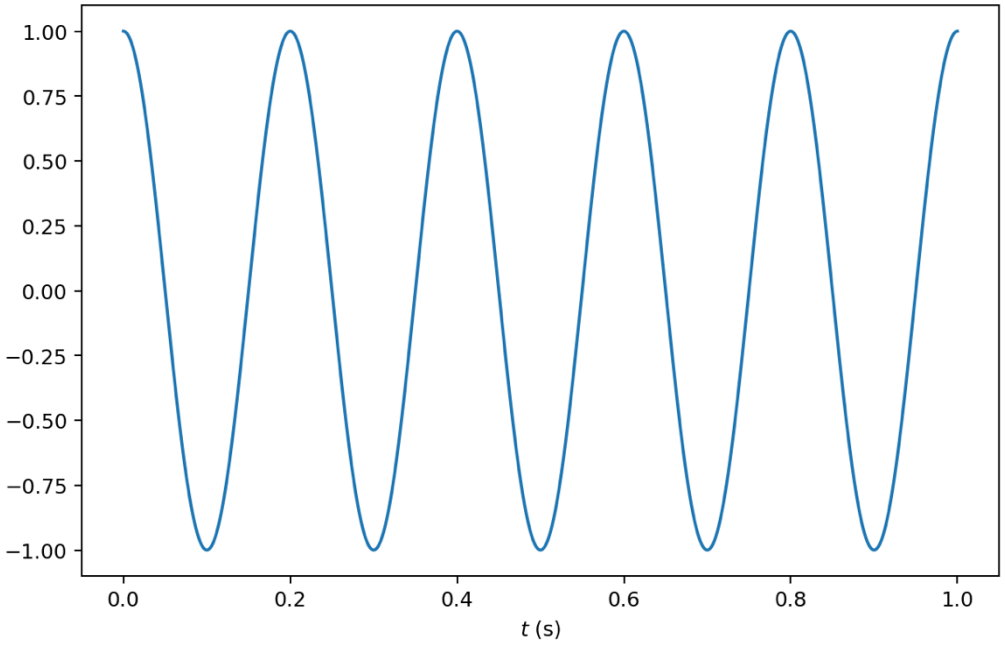
$t \in \mathbb{R}, x(t) \in \mathbb{R}$ $\qquad$ $n \in \mathbb{Z}, x[n] \in \mathbb{R}$

Sampling converts from **analog** to **discrete time**:

$$x[n] = x(nT)$$

T is referred to as the **sampling period**

Sampling Example

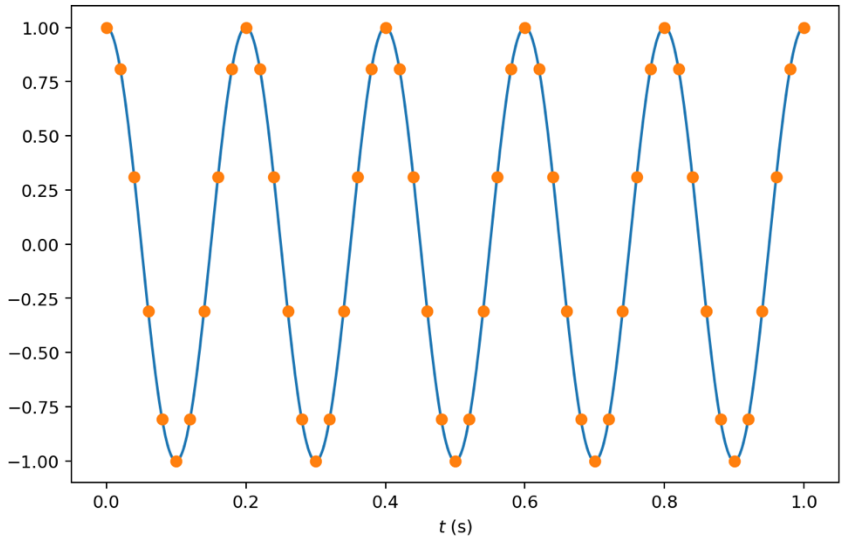


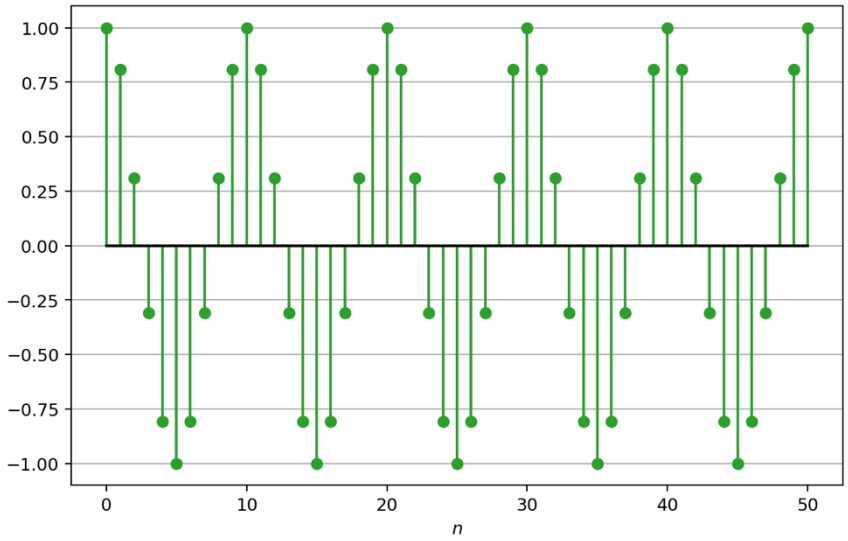
Analog signal: $x(t) = A\cos(2\pi f_0 t)$

What are A and f_0?

$A = 1, f_0 = 5$ Hz

Sampling Example





Discrete-time signal: $x[n] = x(nT) = \cos[2\pi(5)Tn]$ What is T?

$$T = \frac{1}{50\ Hz} = 20\ \text{ms}$$

Quantization

$$x[n] \longrightarrow \boxed{Q} \longrightarrow x_d[n]$$

$n \in \mathbb{Z},\ x[n] \in \mathbb{R}$ $\qquad$ $n \in \mathbb{Z},\ x_d[n] \in \mathbb{Q}$

Quantization converts from **discrete time** to **digital**

Uniform quantizer input/output relationship:

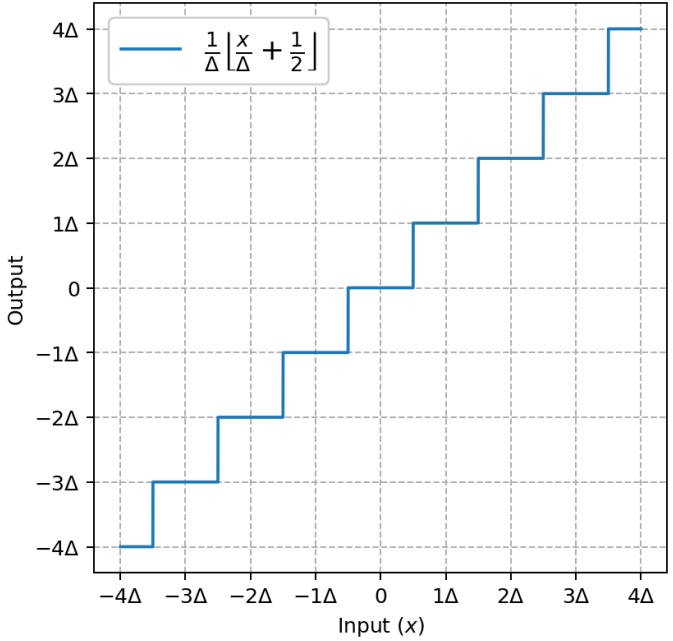
$$x_d[n] = \Delta \left\lfloor \frac{x[n]}{\Delta} + \frac{1}{2} \right\rfloor$$

( )

Δ is referred to as the quantizer **step size**

Uniform Quantizer

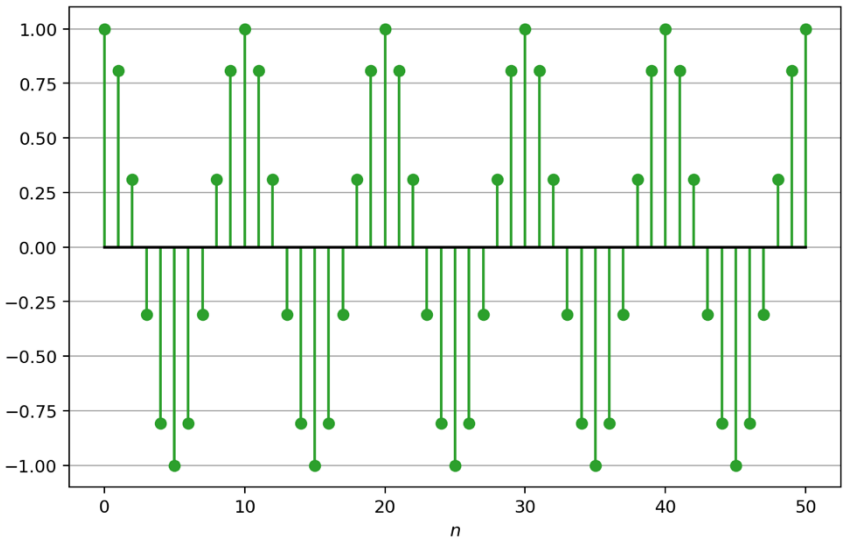
Uniform Quantizer Input to Output Relationship

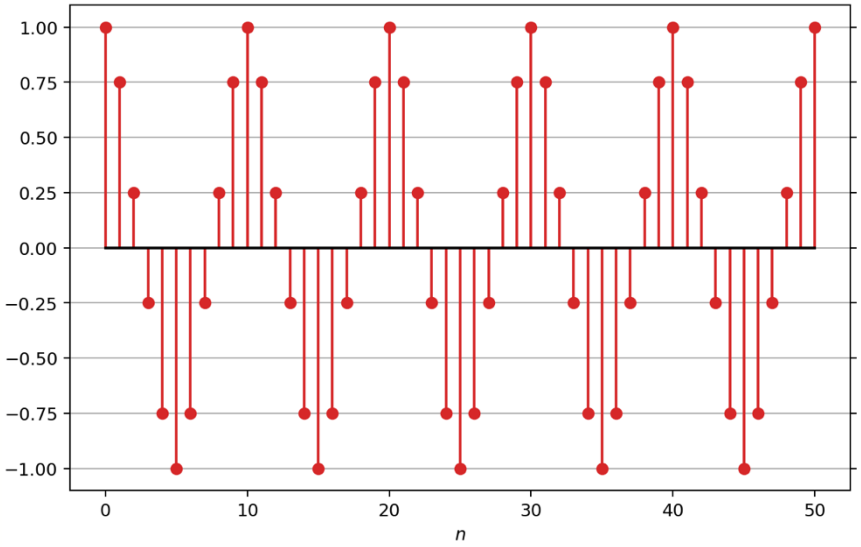


A Value to 8-Bit Mapping

Signal Value	Bits
-4Δ	000
-3Δ	001
-2Δ	010
$-\Delta$	011
0	100
Δ	101
2Δ	110
3Δ	111

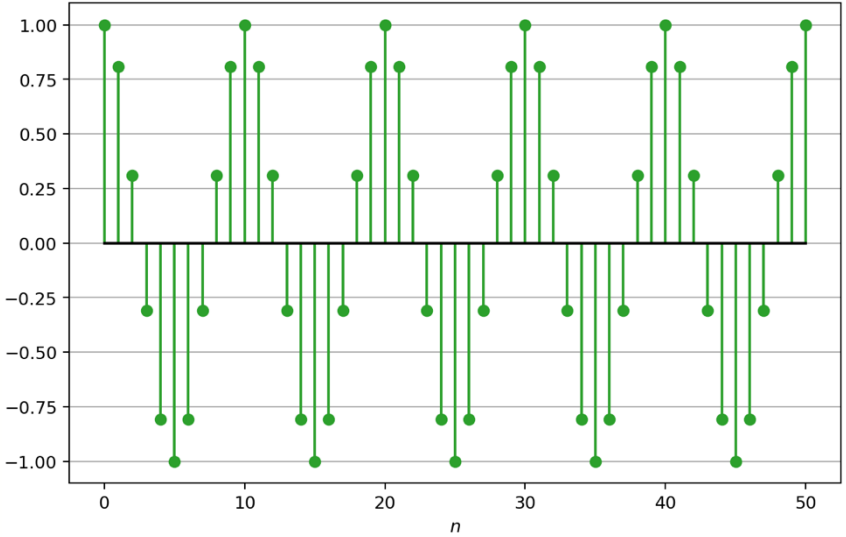
Quantization Example

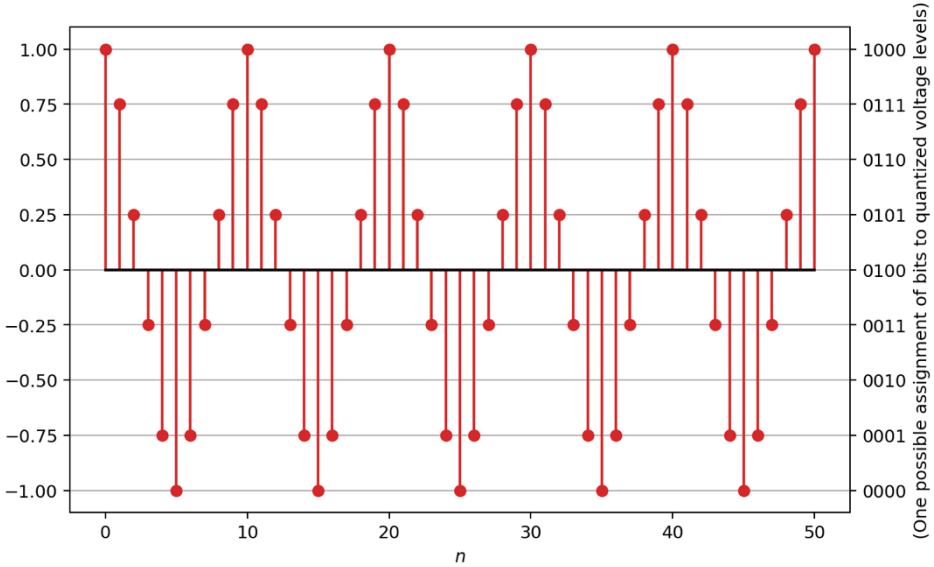




Digital signal: $x_d[n] = \Delta \left\lfloor \frac{\cos\left[2\pi(5)\left(\frac{1}{50}\right)n\right]}{\Delta} + \frac{1}{2} \right\rfloor$ with $\Delta = 0.25$ V

Quantization Example



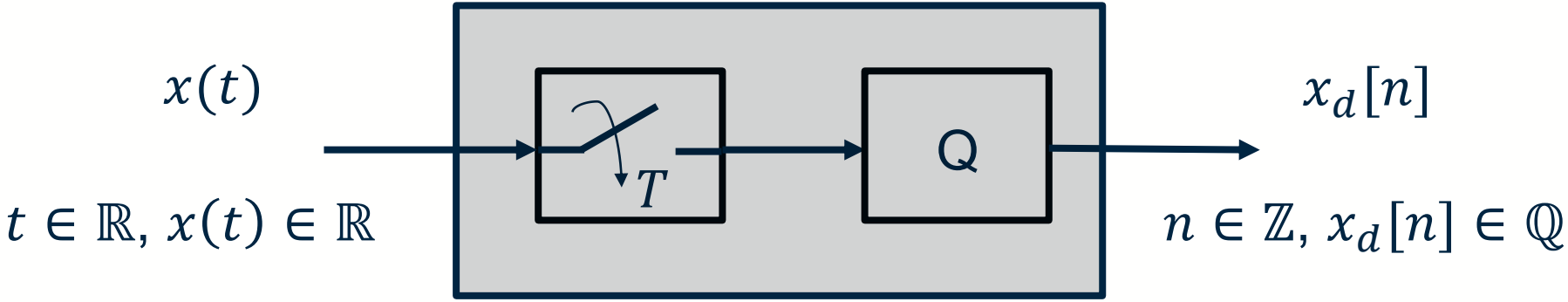


Digital signal: $x_d[n] = \Delta \left\lfloor \frac{\cos\left[2\pi(5)\left(\frac{1}{50}\right)n\right]}{\Delta} + \frac{1}{2} \right\rfloor$ with $\Delta = 0.25$ V

9 quantization levels $\Longrightarrow$ at least 4 bits to store each sample value

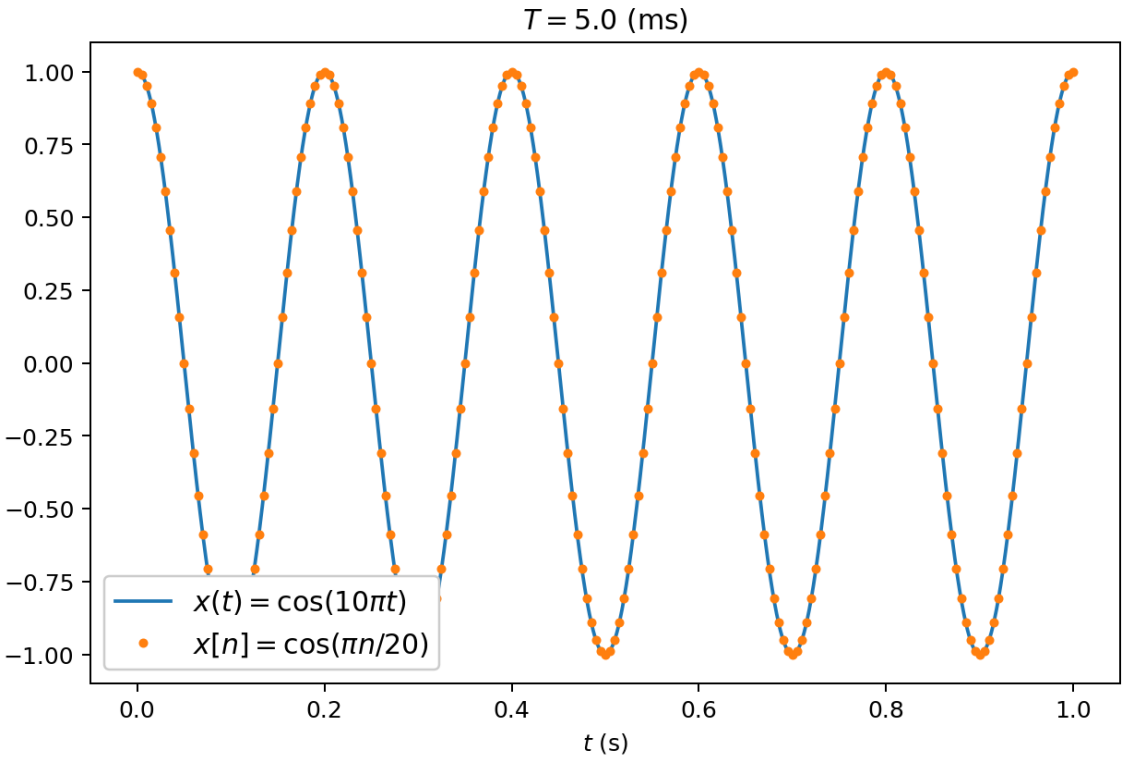
A/D Trade-Off Space

Analog to Digital Converter (A/D, ADC)



- How to choose the sampling period T?
- How to choose quantizer step size Δ?

High Sampling Frequency

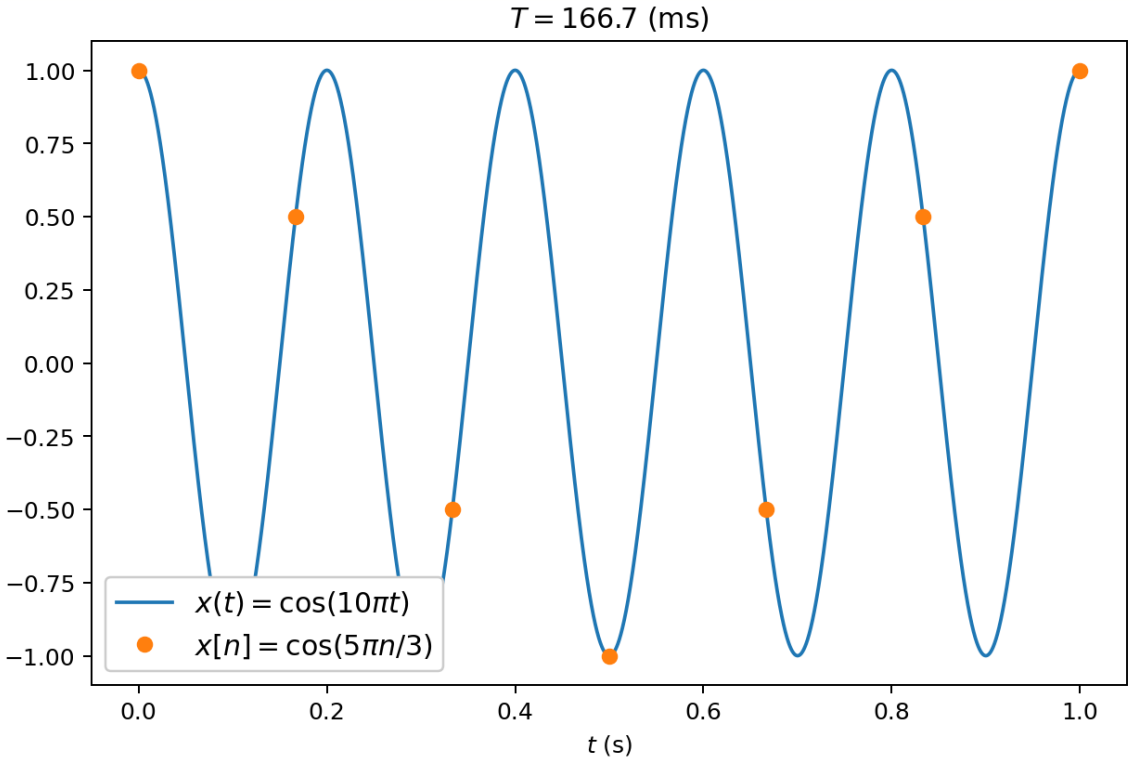


In this case, the samples approximate the signal well

High sampling frequency (small T) comes with a cost:

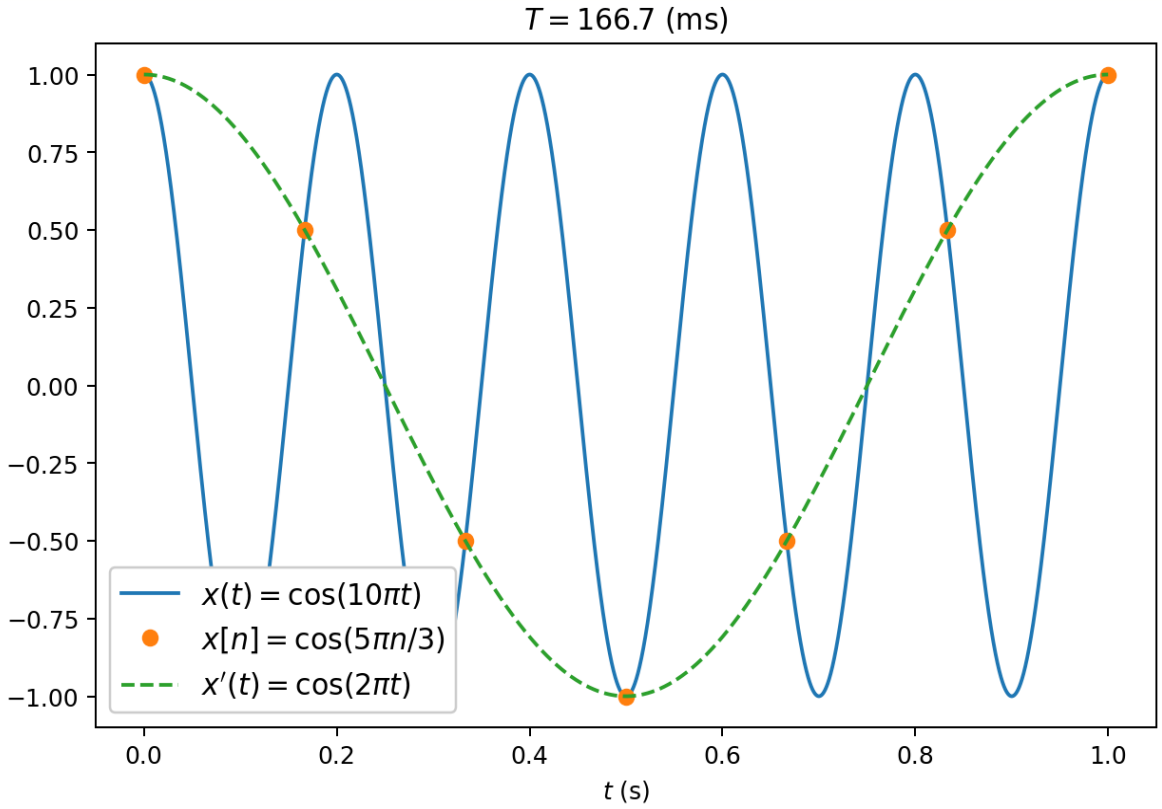
- More memory
- More computation

Low Sampling Frequency



In this case, there is no hope to recover the signal from its samples

Aliasing



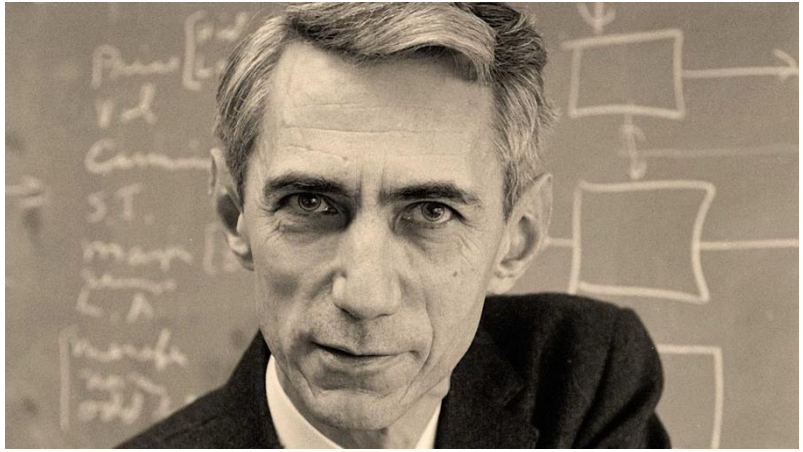
In fact, the samples seem to originate from a lower frequency signal than the sampled signal

This is called **aliasing**

Nyquist-Shannon Sampling Theorem

If a signal contains no frequencies higher than B Hz, then it can be ***perfectly*** reconstructed (no loss of information) from its samples if $T < \frac{1}{2B}$

(Harry Nyquist)



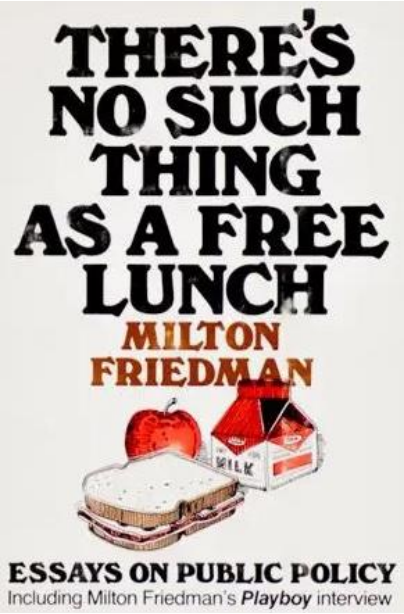
(Claude Shannon)

Quantization Error
Noise

- Smaller quantizer step size Δ (equivalently, larger effective number of bits) ⇔ smaller quantization error

- Quantization error is modeled as noise, i.e., random signal distortion (beyond the scope of UESTC2004 🌶️🌶️)

- Small Δ comes with a cost: more memory and computation



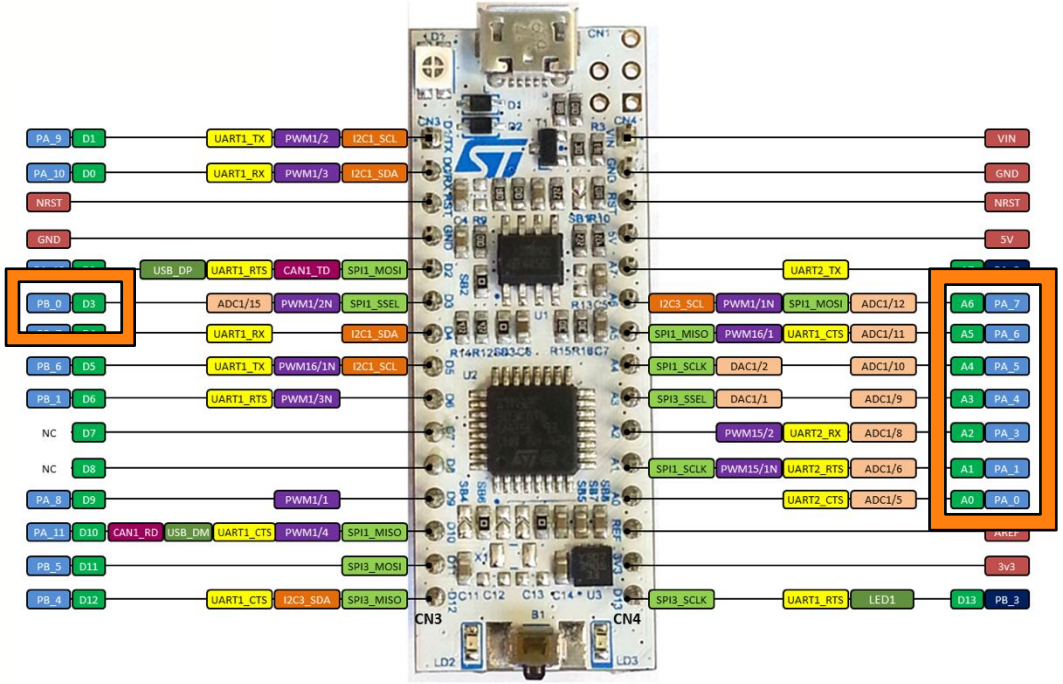
STM32L4 Series ADC Summary

From Chapter 16 of the Reference Guide (RM0394)

- Two ADCs that can be configured for 12, 10, 8, or 6-bit operation
- Maximum (@12-bit operation) of 5.33 mega samples / second (Msps) or 10 Msps by interleaving both ADCs
- Oversampling capability to lower noise (and approximate 16-bit operation). (🌶️🌶️)
- Adaptive sampling speed mode for low-power operation (🌶️🌶️)

NUCLEO L432KC ADC Pins

GPIO pin D3



GPIO pins A0 through A6

- The dev board has 8 input pins that connect to an ADC

NUCLEO L432KC Board – AnalogIn Class Reference

Function	Description
AnalogIn name(PinName pin)	Create an AnalogIn object, connected to the specified pin
float read()	Read the input voltage, represented as a float between (0.0 and 1.0)
unsigned short read_u16()	Read the input voltage, represented as an unsigned short (0x0 – 0xFFFF).
float read_voltage()	Read the input voltage in volts (multiply the result of read() by the get_reference_voltage() value)
void set_reference_voltage (float vref)	Sets this AnalogIn instance's reference voltage. The AnalogIn's reference voltage is used to scale the output.
float get_reference_voltage()	Get this AnalogIn instance's reference voltage. Returns a floating-point value representing this AnalogIn's reference voltage, in volt units.

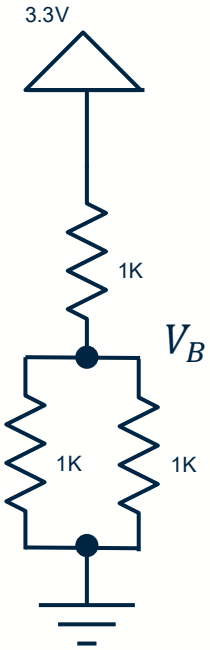
Example: Using the NUCLEO L432KC as a Voltmeter

```cpp
// Based on mbed-os-example-blinky
#include "mbed.h"

// Measurement rate in milliseconds
#define POLL_RATE     200ms

int main() {
    // Initialize the analog pin A0 as an input
    AnalogIn   ain(A0);
    ain.set_reference_voltage(3.3);

    while (1) {
        // Output to serial port (see mbed_app.json for config)
        printf("percentage:  %3.3f%%\n", ain.read() * 100.0f);
        printf("raw value:   0x%04X\n", ain.read_u16());
        printf("voltage:     %3.3f (V)\n", ain.read_voltage());
        printf("\n");

        ThisThread::sleep_for(POLL_RATE);
    }
    return 0;
}
```

Of note:

- This is C++ code (we do not expect you to become experts)
- Infinite loop: get value and print
- Output from printf is “magically” (🤦‍♀️) transmitted via USB

Do the Measured Voltages Make Sense? (🌶️)



- Recall the voltage divider rule:

$$V_{out} = \left(\frac{R_2}{R_1 + R_2}\right) V_{in}$$

- Case 1: $R_1 = R, R_2 = R$

$$V_A = V_{in}/2$$

- Case 2: $R_1 = R, R_2 = R/2$

$$V_b = V_{in}/3$$

Mbed Interrupts

- A quick recap:
  - *Interrupts* allow the microcontroller to respond immediately to events without wasting processing cycles. Interrupts are well-suited for tasks that require **real-time responsiveness** and **minimum latency** in handling critical events like, e.g., monitoring a patient's temperature variations.

  - *Polling* is better suited for low-priority tasks where the real-time aspect is not important.
    - Two main problems with polling:
      - The processor cannot perform any other operations during a polling routine.
      - All inputs are treated as equal; the urgent change has to wait its turn before it is recognised by the microcontroller.
  - You can get questions in the exam presenting you with a scenario and asking you is it better to use interrupts or polling, and why!

Polling

Mbed Interrupts
External Interrupt

- External Interrupt on the mbed:

```
/* Program Example 9.1: Simple interrupt example. External input causes interrupt,
while led flashes
*/
#include "mbed.h"
#include "PinNamesTypes.h"
DigitalOut led(LED1);
DigitalOut flash(D9);

InterruptIn button(A6, PullDown);  // name the interrupt input

void ISR1() { //this is the response to interrupt, i.e. the ISR
    led = !led;
}


int main() {


    button.rise(&ISR1); // attach the address of the ISR function to the


// interrupt rising edge
    while(1) { // continuous loop, ready to be interrupted
        flash = !flash;
    }
}
```

Create an InterruptIn connected to the specified pin, and the pin configured to the specified mode

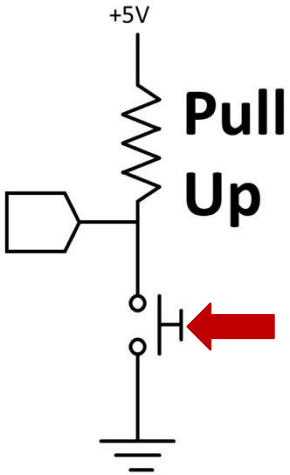
Interrupt Service Routine (ISR), i.e., response to the interrupt

Trigger on rising edge and execute "ISR1"

Main program that will be interrupted

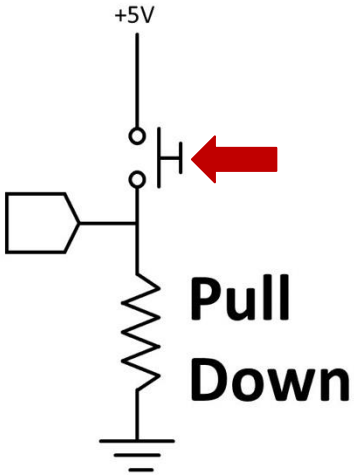
Mbed Interrupts
External Interrupt

- Pull Up Resistor:



- **Push**: HIGH to LOW (**falling edge**)
- **Release**: LOW to HIGH (**Rising edge**)

- Pull-Down Resistor:



- **Push**: LOW to HIGH (**Rising edge**)
- **Release**: HIGH to LOW (**falling edge**)

Mbed Interrupts
External Interrupt & ADC

- Event-based Analogue-to-Digital (AD) conversion, e.g., manual temperature reading.
- The code implements an externally-triggered ADC conversion.

```c
#include "mbed.h"

#define EXT_ADC_Trigger
AnalogIn Ain(D3); // defines analogue in on pin D3

#ifdef EXT_ADC_Trigger
    InterruptIn button(A6);
    volatile bool conversionTriggered = false;  // Flag to indicate if ADC conversion is triggered
    volatile float analogValue = 0.0f;  // Variable to store analog value
    int print_flag = 0; //A flag to avoid continuous orinting of the same message
    // Button interrupt handler

    void button_isr() {
        printf("EXT Interrupt ISR\n\r");
        conversionTriggered = true;  // Set flag to trigger ADC conversion
        print_flag = 0;
    }
#endif
```

```c
int main(){

    printf("Main Program Running\n\r");

    #ifdef EXT_ADC_Trigger
        button.mode(PullDown);
        button.rise(&button_isr);  // Attach falling edge interrupt for button press
    #endif
    while (1){

        #ifdef EXT_ADC_Trigger
            if (conversionTriggered) {
            // Start ADC conversion
            analogValue = Ain.read();  // Trigger ADC conversion and directly read the value
            printf("Analog Value: %.2f V\n\r", analogValue*3.3);  // Print analog value
            conversionTriggered = false;  // Reset flag
            //wait(1);
            }
            else if(conversionTriggered == false && print_flag == 0)
            {
                printf("ADC Conversion not yet triggered by Ext Interrupt!\n\r");
                print_flag = 1;
            }
        #endif
    }//end while
}
```

Trigger on rising edge and execute "button_isr"

Output

Problems × Output × Debug Console × >_ NUCLEO-L432KC ×

```
Main Program Running
ADC Conversion not yet triggered by Ext Interrupt!
EXT Interrupt ISR
Analog Value: 2.19 V
ADC Conversion not yet triggered by Ext Interrupt!
EXT Interrupt ISR
Analog Value: 2.19 V
ADC Conversion not yet triggered by Ext Interrupt!
```

ADC Triggered

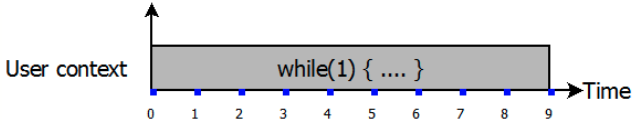
Rising Edge – What does this mean if Pull-Up or Pull-Down?

Mbed Interrupts
Timeout Interrupt

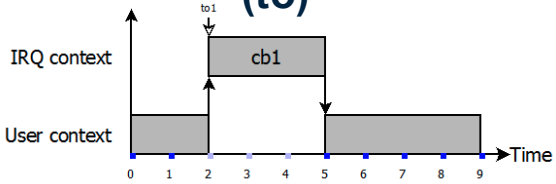
- Use the *Timeout* interface to set up an interrupt to call a function, ***once***, after a specified delay.

```
#include "mbed.h"

Timeout to1;
DigitalOut led1(LED1);
DigitalOut led2(A4);

//Timeout ISR
void cb1(void) {
    led2 = 1;
    wait_us(3000000);
    led2 = 0;
}

int main() {

    to1.attach(&cb1, 2);//Timeout after 2 seconds

    while(1) {
        led1 = 1;
        wait_us(50000);
        led1 = 0;
        wait_us(950000);
    }
}
```

Without the Timeout (to)



With the Timeout (to)

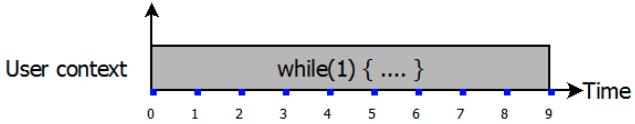


Timeout (to)

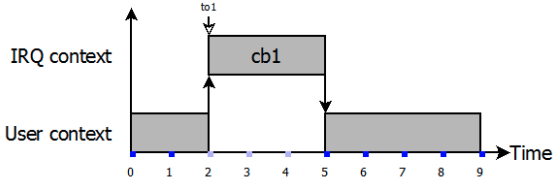
Mbed Interrupts
Timeout Interrupt

```
#include "mbed.h"
#include "mbed_wait_api.h"

DigitalOut led1(LED1);
DigitalOut led2(A4);
DigitalOut led3(D12);

Timeout to1;
Timeout to2;

//Timeout 1 ISR
void cb1(void) {
    led2 = 1;
    wait_us(3000000);
    led2 = 0;
}

//Timeout 2 ISR
void cb2(void) {
    led3 = 1;
    wait_us(3000000);
    led3 = 0;
}

int main() {

    to1.attach(&cb1, 2); //Timeout after 2 seconds
    to2.attach(&cb2, 3); //Timeout after 3 seconds

    while(1) {
        led1 = 1;
        wait_us(50000);
        led1 = 0;
        wait_us(950000);
    }
}
```

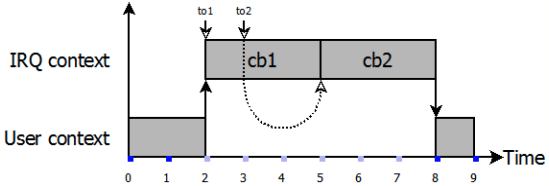
Without the Timeout (to)



With one Timeout (to1)



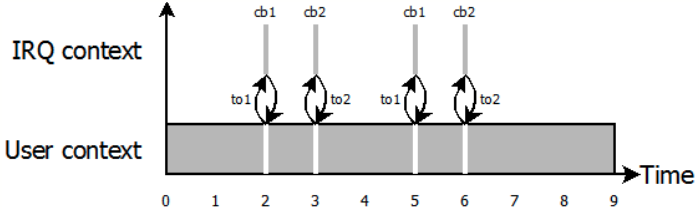
With two Timeouts (to1 & to2)



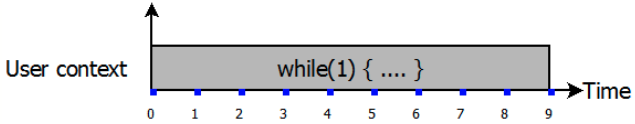
Timeout (to)

Mbed Interrupts
Timeout Interrupt – Formative Assessment (FA1)

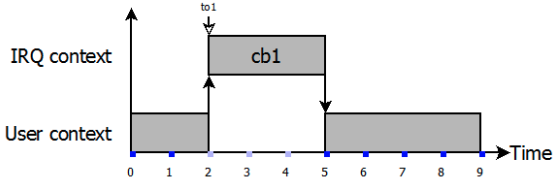
- A rule of thumb: Don't use *wait()* in ISR call backs (*cb*).
- The implementation we made was for the illustration of the concept only.
- **You are now required to write a code that implements two timeouts to achieve the following behaviour:**
  - led1: flash brief one per second, EVERY second.
  - led2: On at t = 2 and off at t = 5
  - led3: On at t = 3 and off at t = 6



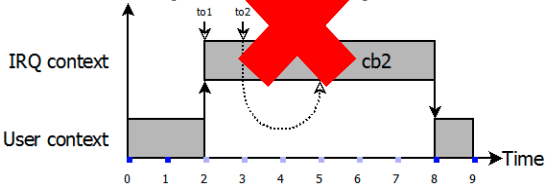
Without the Timeout (to)



With one Timeout (to1)



With two Timeouts (to1 & to2)



Timeout (to)

Mbed Interrupts
Timeout Interrupt for Debouncing

- Mechanical push buttons tend to "bounce”! When you press or release them, the contact can rapidly oscillate between high and low for a few milliseconds. This results in multiple interrupts being fired when you're only expecting one.

- How can a Timeout help? – Yes!!!
  - Disable further interrupts immediately after the first press.
  - Start a Timeout timer for the expected bounce duration (e.g. 10–50 ms).
  - After the timeout, re-enable the interrupt.

Timeout (to)

Mbed Interrupts
Timeout Interrupt for Debouncing

- *Timeout* interrupt can be very useful in *debouncing*!

```
#include "mbed.h"

AnalogIn Ain(D3); // defines analogue in on pin D3

Timeout debounceTimeout;
InterruptIn button(A6);

volatile bool conversionTriggered = false;  // Flag to indicate if ADC conversion is triggered
volatile float analogValue = 0.0f;  // Variable to store analog value
volatile bool debounceInProgress = false; //flag to idnicate if debouncing is in progress
int print_flag = 0; //Flag to control the printing of debugging messages

// Function prototypes
void button_isr();
void button_debounced();

// Button interrupt handler
void button_isr() {
    if (!debounceInProgress) {
        debounceInProgress = true;
        conversionTriggered = true;  // Set flag to trigger ADC conversion
        print_flag = 0;
        // Disable the interrupt temporarily to debounce
        button.rise(NULL); // Remove the handler

        // Start the debounce timeout (50 ms)
        debounceTimeout.attach(&button_debounced, 0.1);// Debounce time: 100 milliseconds
    }
}

void button_debounced() {
    // Button press has been debounced; perform desired action
    // This function will be called after the debounce period
    printf("Debouncing ISR\n\r");
    debounceInProgress = false;
    button.rise(&button_isr);
}
```

Trigger "TimeOut"

Reset Debounce

Two Interrupts:

- **External**
- **TimeOut**

Timeout (to)

Mbed Interrupts
Timeout Interrupt for Debouncing

```
int main(){

    printf("Main Program Running\n\r");
    printf("This example is running on Mbed OS sd.&d.&d.\n\r",
    MBED_MAJOR_VERSION, MBED _MINOR_VERSION, MBED_PATCH_VERSION);
    button.mode(PullDown);
    button.rise(&button_isr);
    while (1){
        if (conversionTriggered) {
            // Start ADC conversion
            analogValue = Ain.read();  // Trigger ADC conversion and directly read the value
            printf("Analog Value: %.2f V\n\r", analogValue*3.3);  // Print analog value
            conversionTriggered = false;  // Reset flag
            //wait(1);
        }
        else if(conversionTriggered == false && print_flag == 0)
        {
            printf("ADC Conversion not yet triggered by Ext Interrupt (with Timeout)!\n\r");
            print_flag = 1;
        }
    }//end while
}/*
```

Since this is a "Timeout" is the debouncing going to happen <u>once</u> only?

Timeout (to)

Output

Timeout (Debouncing) Triggered

```
Problems ×   Output ×   Debug Console ×   >_ NUCLEO-L432KC ×

Main Program Running
This example is running on Mbed OS 6.13.0.
ADC Conversion not yet triggered by Ext Interrupt (with Timeout)!
Analog Value: 2.19 V
ADC Conversion not yet triggered by Ext Interrupt (with Timeout)!
Debouncing ISR
```

ADC Triggered

Mbed Interrupts
Ticker Interrupt

- Use the *Ticker* interface to set up an interrupt to call a function, ***periodically***, after a specified delay.

```
#include "mbed.h"

Ticker flipper;
DigitalOut led1(LED1);
DigitalOut led2(A4);

void Tick()
{
    led2 = !led2;
}

int main()
{
    led2 = 1;
    flipper.attach(&Tick, 2.0); // the address of the function to be attached (flip) and the interval (2 seconds)

    // spin in a main loop. flipper will interrupt it to call flip
    while (1) {
        led1 = !led1;
        ThisThread::sleep_for(200);
    }
}
```

(Tick)

Tick

Ticker

Mbed Interrupts
Ticker Interrupt & ADC

- Let us use the *Ticker* interrupt to ***periodically*** read an analogue input.

```
#include "mbed.h"
#define Ticker_Interrupt

AnalogIn Ain(D3); // defines analogue in on pin D3

#ifdef Ticker_Interrupt
    Ticker Periodic_Read;
    int print_flag = 0;
    volatile float analogValue = 0.0f;  // Variable to store analog value
    volatile int Tick = 0;

    void Periodic_ADC_Sample() {
        printf("Ticker Interrupt ISR\n\r");
        Tick = 1;
        print_flag = 0;//Limit the print to once per trigger
}
#endif
```

Ticker

Mbed Interrupts
Ticker Interrupt & ADC

```c
int main(){

    printf("Main Program Running\n\r");
    printf("This example is running on Mbed OS %d.%d.%d.\n\r",
    MBED_MAJOR_VERSION, MBED_MINOR_VERSION, MBED_PATCH_VERSION);

    #ifdef Ticker_Interrupt
        Periodic_Read.attach(&Periodic_ADC_Sample, 5.0); // setup flipper to call flip after 2 seconds
    #endif
    while (1){

        #ifdef Ticker_Interrupt
            if (Tick) {
            // Start ADC conversion
            analogValue = Ain.read();  // Trigger ADC conversion and directly read the value
            printf("Analog Value: %.2f V\n\r", analogValue*3.3);  // Print analog value
            Tick = 0;  // Reset flag
            //wait(1);
            }
            else if(Tick == 0 && print_flag == 0)
            {
                printf("ADC Conversion not yet triggered by Ticker!\n\r");
                print_flag = 1;
            }
        #endif

    }//end while
}
```

Ticker

Output

2nd ADC Trigger After Another 2 Seconds

```
Problems × Output × Debug Console × NUCLEO-L432KC ×

Main Program Running
This example is running on Mbed OS 6.13.0.
ADC Conversion not yet triggered by Ticker!
Ticker Interrupt ISR
Analog Value: 2.19 V
ADC Conversion not yet triggered by Ticker!
Ticker Interrupt ISR
Analog Value: 1.31 V
```

1st ADC Trigger After 2 Seconds

University of Glasgow

University of Glasgow 2013 Glasgow College, UESTC

WORLD CHANGING GLASGOW

Summary

- Analog to digital conversion
- ADC tradeoff space
- ADC mbed implementation
- Introduction to mbed interrupts
- Implementation of External, Timeout, and Ticker interrupts on the mbed with ADC.